

Rank 5th Solution

Rank	4
Team	yanqiangmiffy
Author	yanqiangmiffy
Topic ID	670668
Version	Original

Source: <https://www.kaggle.com/competitions/csiro-biomass/discussion/670668>

Retrieved with Kaggle CLI on 2026-07-18.

CSIRO Image2Biomass Prediction - 5th Place Solo Gold Medal Solution

First of all, thanks to Kaggle and CSIRO for organizing this competition. This is a very meaningful agricultural AI application scenario.

1. Introduction

This is my first time participating in a Kaggle CV competition, and I'm very happy and fortunate to have achieved 5th place solo (Private LB: 0.76). Throughout the competition, I conducted extensive experiments - **I did about 100+ training experiments** (from exp-v1 to exp-v188). Through this competition, I learned a lot about computer vision. Below is a detailed sharing of my competition solution.

Submissions evaluated for final score

All		Successful	Selected	Errors	Private Score		
Submission and Description		Private Score		Public Score	Selected		
✓	exp177-csiro-inference-cv - Version 1 Succeeded · 7d ago · v164+调整损失权重, 加大损失权重 total	0.66	0.75	☐			
✓	CSIRO-Ensemble-Models-V11 - Version 26 Succeeded · 2d ago · 3tta+v177+148v2	0.66	0.76	☐			
✓	exp185-csiro-inference-cv - Version 1 Succeeded · 2d ago · exp-v185 0.9674 0.xx v181+ng segmentation_loss_weight 0.3	0.66	0.74	☐			
✓	CSIRO-Ensemble-Models-V9 - Version 8 Succeeded · 2d ago · 相比v6替换为exp148v2	0.66	0.76	☐			
✓	CSIRO-Ensemble-Models-V12 - Version 7 Succeeded · 1d ago · 根据训练集设置阈值	0.66	0.76	☒			
✓	CSIRO-Ensemble-Models-V15 - Version 1 Succeeded · 12h ago · 142 two tta,在CSIRO-Ensemble-Models-V12 - Version 7基础上 (0.76)	0.66	0.76	☐			
✓	exp181-csiro-inference-cv - Version 2 Succeeded · 4d ago · v174+unet dinov3 unet 1024 ng	0.66	0.74	☐			
✓	CSIRO-Ensemble-Models-V9 - Version 6 Succeeded · 3d ago · two tta调整权重	0.66	0.76	☐			
✓	CSIRO-Ensemble-Models-V9 - Version 3 Succeeded · 6d ago · two tta	0.65	0.76	☐			
✓	exp146-csiro-inference-cv - Version 3 Succeeded · 14d ago · dead/total ratio预测	0.65	0.74	☐			
✓	CSIRO-Ensemble-Models-V10 - Version 4 Succeeded · 5d ago	0.65	0.76	☐			
✓	exp164-csiro-inference-cv - Version 2 Succeeded · 9d ago · exp-v164 0.9764 0.71 0.75 v147+使用多种数据增强方式进行训练	0.65	0.75	☐			
✓	CSIRO-Ensemble-Models-V16 - Version 9 Succeeded · 9h ago · lgb online	0.65	0.76	☒			

2. Competition Understanding

2.1 Problem Description

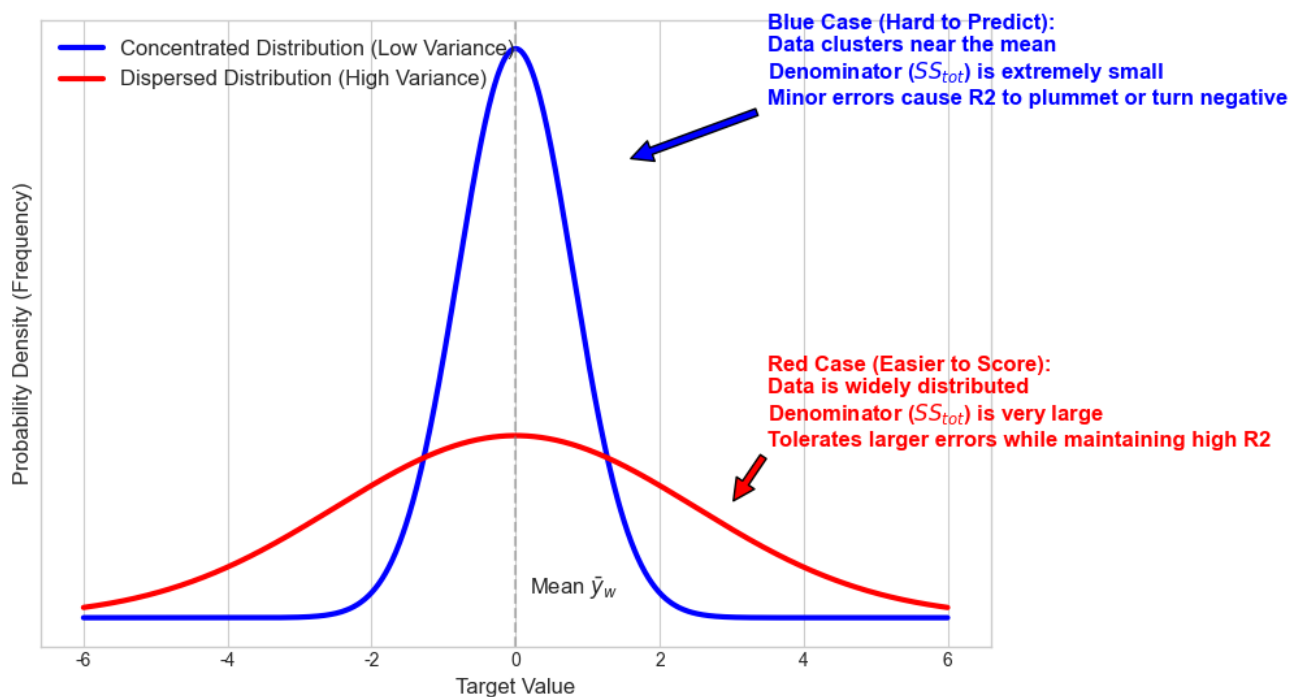
The goal of this competition is to use pasture images to predict five key biomass components:

- **Dry_Green_g** - Dry green vegetation (excluding clover)
- **Dry_Dead_g** - Dry dead material
- **Dry_Clover_g** - Dry clover biomass
- **GDM_g** - Green dry matter
- **Dry_Total_g** - Total dry biomass

2.2 Data Characteristics

- **Small training set:** Approximately 800+ images
- **Image size:** Original images are 2000x1000 pixels
- **Distribution shift:** Significant differences between training and test set distributions
- **Multi-state data:** Data from different Australian states (NSW, Tas, Vic, WA)
- **Physical constraint relationships:** Physical relationships exist between targets
 - Dead = Total - GDM
 - Clover = GDM - Green

Trend of R^2 Metric Relative to Data Concentration



2.3 Evaluation Metric

The competition uses weighted R^2 score as the evaluation metric, with weighted average of R^2 across five targets.

3. Core Improvement Techniques

After extensive ablation experiments, the following are key techniques that can stably improve scores:

3.1 Image Preprocessing - Timestamp Removal

Original images contain orange date timestamps, which are data noise. I used HSV color space detection and image inpainting techniques to remove these timestamps:

```
def clean_image(img):  
    """  
    Image preprocessing: Remove bottom artifacts and date stamp.  
    """  
    # Detect orange date stamp in HSV space  
    hsv = cv2.cvtColor(img, cv2.COLOR_RGB2HSV)  
    # Define orange range  
    lower = np.array([5, 150, 150])  
    upper = np.array([25, 255, 255])  
    mask = cv2.inRange(hsv, lower, upper)  
  
    # Dilate mask to cover text edges  
    mask = cv2.dilate(mask, np.ones((3, 3), np.uint8), iterations=2)  
  
    # Inpaint if timestamp detected  
    if np.sum(mask) > 0:  
        img = cv2.inpaint(img, mask, 3, cv2.INPAINT_TELEA)  
  
    return img
```



+ Code + Markdown

[3]:

3.2 Multiple Data Augmentations

Due to the small training set size, data augmentation is crucial for preventing overfitting. I used TTA-style training strategy:

```
def get_tta_transforms(img_size: int) -> list[A.Compose]:
    """Returns a list of transform pipelines for TTA-style training."""
    normalize = A.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )

    # View 0: Original
    original_view = A.Compose([
        A.Resize(img_size, img_size),
        normalize,
        ToTensorV2()
    ])

    # View 1: Horizontal Flip
    hflip_view = A.Compose([
        A.Resize(img_size, img_size),
        A.HorizontalFlip(p=1.0),
        normalize,
        ToTensorV2()
    ])

    # View 2: Vertical Flip
    vflip_view = A.Compose([
        A.Resize(img_size, img_size),
        A.VerticalFlip(p=1.0),
        normalize,
        ToTensorV2()
    ])

    # View 3: Rotate 90 degrees
    rotate90_view = A.Compose([
        A.Resize(img_size, img_size),
        A.Rotate(limit=(90, 90), p=1.0, border_mode=0),
        normalize,
        ToTensorV2()
    ])

    return [hflip_view, vflip_view, rotate90_view, original_view]
```

Training Strategy:

- First 15 epochs: Train with augmented views (hflip, vflip, rotate90), equivalent to 4x data
- Subsequent epochs: Use only original view
- **Key finding:** Randomly selecting augmentation methods works better than sequential epoch-based selection (exp-v174)

3.3 Multi-View Input Architecture

In addition to splitting the image into left and right halves (simulating stereo vision), I also added the full image as a third input stream:

```
class BiomassDataset(Dataset):
    """Three-stream dataset: splits 2000x1000 image into left, right, and full."""

    def __getitem__(self, idx: int):
        image = self._load_image(self.image_paths[idx])
        image = clean_image(image)

        height, width = image.shape[:2]
```

```

mid = width // 2
left = image[:, :mid]      # Left half
right = image[:, mid:]     # Right half
full_image = image.copy()  # Full image

# Apply same transform to all three views
left = self.transform(image=left)["image"]
right = self.transform(image=right)["image"]
full_image = self.transform(image=full_image)["image"]

return left, right, full_image, train_tgt, eval_tgt, species_tgt

```

This design allows the model to learn both local details (left/right views) and global context (full view) simultaneously.

3.4 Mixture-of-Experts (MoE) Model Architecture

Design Intent: The model can automatically switch to the most suitable expert for prediction based on the specific characteristics of the input image (e.g., more green grass or more dead grass), effectively resolving feature conflicts in multi-task learning.

```

class MetaMoE(nn.Module):
    """Mixture-of-Experts head for main targets, outputs mean and variance."""

    def __init__(self, in_dim: int, hidden: int, num_experts: int,
                  out_dim: int = 1, dropout: float = 0.2):
        super().__init__()
        self.num_experts = num_experts

        # Gating network
        self.gate = nn.Sequential(
            nn.LayerNorm(in_dim),
            nn.Linear(in_dim, 256),
            nn.ReLU(),
            nn.Linear(256, num_experts),
        )

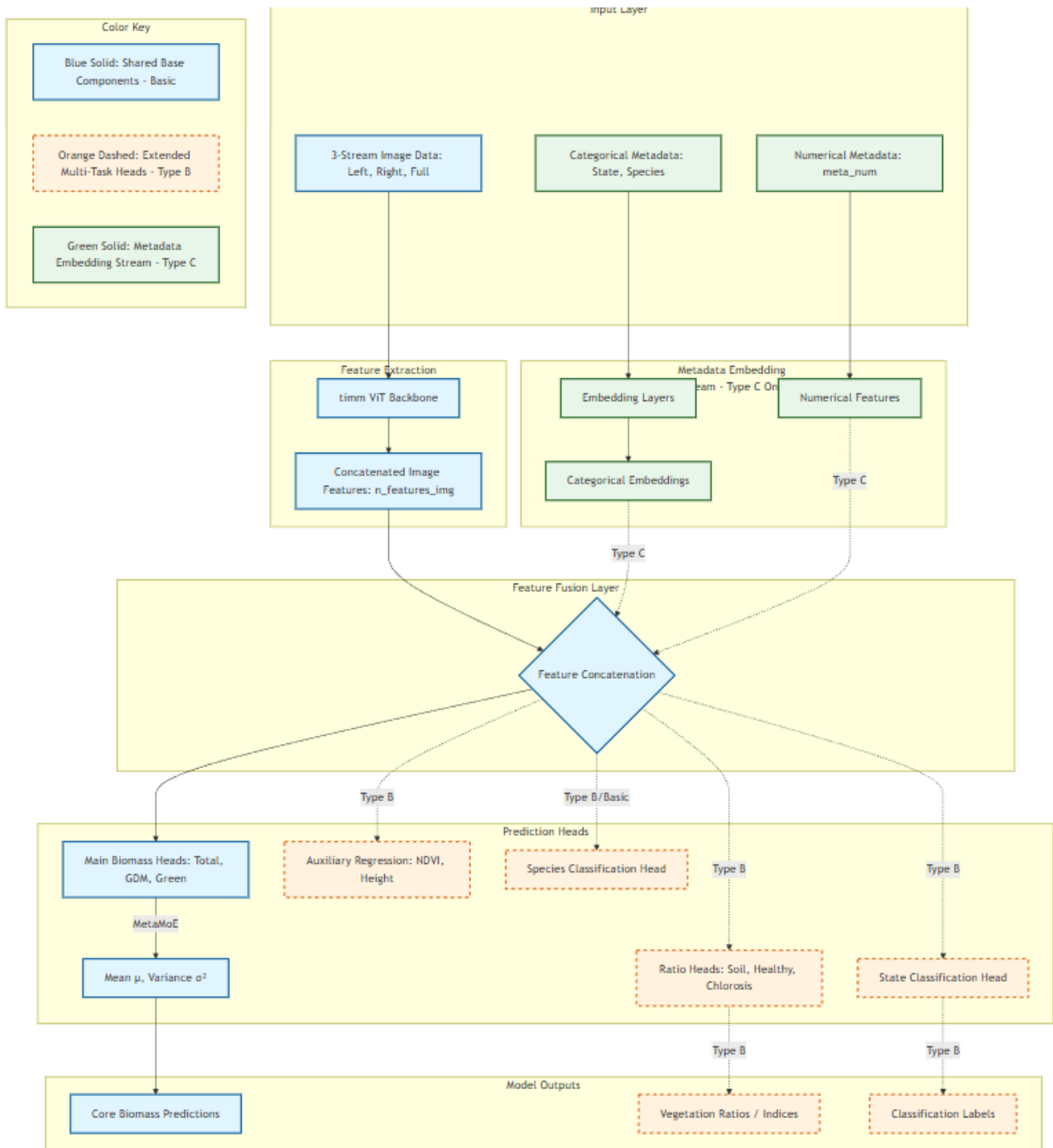
        # Expert networks (each outputs mean and variance)
        self.experts = nn.ModuleList([
            nn.Sequential(
                nn.LayerNorm(in_dim),
                nn.Linear(in_dim, hidden),
                nn.ReLU(),
                nn.Dropout(dropout),
                nn.Linear(hidden, 256),
                nn.ReLU(),
                nn.Linear(256, 2), # [mean, var_logit]
            )
            for _ in range(num_experts)
        ])

    def forward(self, x: torch.Tensor):
        gate_logits = self.gate(x)
        gate_weights = F.softmax(gate_logits, dim=1).unsqueeze(-1)
        expert_outs = torch.stack([expert(x) for expert in self.experts], dim=1)
        out = (expert_outs * gate_weights).sum(dim=1)

        pred_mean = out[:, 0:1]
        pred_var_logit = out[:, 1:2]
        pred_var = F.softplus(pred_var_logit) + 1e-6 # Ensure variance is positive

```

```
return pred_mean, pred_var
```



Three Model Variants:

- **Model Type A (BiomassModelBasic):** Basic three-stream model (exp-v147, exp-v164, exp-v174)
- **Model Type B (BiomassModelWithStateSpecies):** With State/Species classification heads (exp-v142)
- **Model Type C (BiomassModelWithMetaEmbedding):** With meta feature embeddings (exp-v148)

3.5 Auxiliary Training Heads

Adding extra training objectives helps the model learn richer feature representations:

```

class BiomassModel(nn.Module):
    def __init__(self, model_name: str, pretrained: bool = True,
                  pretrained_path: str = '', num_species: int = 0):
        super().__init__()
        # ... backbone initialization ...

        # Main regression heads (using MoE)
        self.head_total = MetaMoE(...)
        self.head_gdm = MetaMoE(...)
        self.head_green = MetaMoE(...)

        # Auxiliary regression heads (NDVI and Height)
        self.head_ndvi = _AuxHead(self.n_features, self.n_features // 2)
        self.head_height = _AuxHead(self.n_features, self.n_features // 2)

        # Optional classification heads
        if num_species > 0:
            self.head_species = self._make_classification_head(num_species)

```

Auxiliary Targets:

- Pre_GSHH_NDVI (Normalized Difference Vegetation Index)
- Height_Ave_cm (Average pasture height)
- Species (Species classification, 15 classes)
- State (State classification, 4 classes)

3.6 Physics Consistency Loss

Designed loss function based on physical relationships between targets, using GaussianNLLLoss as the main loss function:

Core Advantages:

- Addresses the R^2 metric's **sensitivity to outlier errors**
- Automatically downweights high-noise samples via GaussianNLL (avoids being biased by dirty data)
- Uses physical constraints to ensure prediction reasonableness (avoids logical contradictions leading to large errors)

```

class PhysicsConsistencyLoss(nn.Module):
    """
    Physics consistency loss using GaussianNLLLoss

    Key advantages:
    - Automatic outlier handling: model learns to predict high variance for noisy samples
    - Handles heteroscedasticity: different uncertainty for different biomass levels
    - Physical constraints: Dead = Total - GDM, Clover = GDM - Green
    """

    def __init__(self, weights: dict[str, float] = None):
        super().__init__()
        self.w = {
            "total": 0.45,
            "gdm": 0.2,
            "green": 0.1,
            "dead": 0.1,
            "clover": 0.1,
            "ndvi": 0.0,

```



```

        "height": 0.0,
        "species": 0.05
    }
    self.criterion = nn.GaussianNLLLoss()

def forward(self, preds, targets, species_targets=None):
    # Direct prediction losses
    loss_total = self.criterion(pred_total_mean, gt_total, pred_total_var)
    loss_gdm = self.criterion(pred_gdm_mean, gt_gdm, pred_gdm_var)
    loss_green = self.criterion(pred_green_mean, gt_green, pred_green_var)

    # Physics constraint losses
    # Dead = Total - GDM
    pred_dead_mean = pred_total_mean - pred_gdm_mean
    pred_dead_var = pred_total_var + pred_gdm_var
    gt_dead = gt_total - gt_gdm
    loss_dead = self.criterion(pred_dead_mean, gt_dead, pred_dead_var)

    # Clover = GDM - Green
    pred_clover_mean = pred_gdm_mean - pred_green_mean
    pred_clover_var = pred_gdm_var + pred_green_var
    gt_clover = gt_gdm - gt_green
    loss_clover = self.criterion(pred_clover_mean, gt_clover, pred_clover_var)

    # Weighted total loss
    total_loss = (
        self.w["total"] * loss_total +
        self.w["gdm"] * loss_gdm +
        self.w["green"] * loss_green +
        self.w["dead"] * loss_dead +
        self.w["clover"] * loss_clover
    )
    return total_loss

```

3.7 Layer-wise Learning Rate

Set different learning rates for pretrained Backbone and newly initialized Heads:

```

param_groups = []
for name, param in model.named_parameters():
    if 'backbone' in name:
        # Backbone uses smaller lr to preserve pretrained capability
        param_groups.append({
            'params': param,
            'lr': cfg.finetune_lr * cfg.backbone_lr_mult # typically 0.1
        })
    elif 'head' in name:
        # Head uses larger lr for fast fitting
        param_groups.append({
            'params': param,
            'lr': cfg.finetune_lr * cfg.head_lr_mult # typically 1.0
        })
    else:
        param_groups.append({'params': param, 'lr': cfg.finetune_lr})

optimizer = optim.AdamW(param_groups, weight_decay=cfg.weight_decay)

```

Rationale: Set smaller learning rate for pretrained Backbone to preserve its general feature extraction capability (prevent destroying pretrained weights), while setting larger learning rate for newly initialized Heads to quickly fit the current specific task.

3.8 Post-processing Strategies

3.8.1 State-based Post-processing

Analysis revealed that WA state data has different characteristics from other states:

```
# WA state special handling: Dead is 0, Total equals GDM
if predicted_states_df is not None:
    final_pred = final_pred.merge(predicted_states_df, on='image_path', how='left')
    is_wa = final_pred['Predicted_State'] == 'WA'
    final_pred.loc[is_wa, 'Dry_Death_g'] = 0
    final_pred.loc[is_wa, 'Dry_Total_g'] = final_pred.loc[is_wa, 'GDM_g']
```

3.8.2 Statistics-based Range Clipping

Clip results based on training set target value statistics per state:

```
state_stats = {
    'Tas': {'Dry_Clover_g': {'min': 0.0, 'max': 71.79}, ...},
    'NSW': {'Dry_Clover_g': {'min': 0.0, 'max': 10.10}, ...},
    'WA': {'Dry_Clover_g': {'min': 0.0, 'max': 58.88}, ...},
    'Vic': {'Dry_Clover_g': {'min': 0.0, 'max': 67.90}, ...}
}

for state in state_stats.keys():
    state_mask = final_pred['Predicted_State'] == state
    for var in target_vars:
        min_val = state_stats[state][var]['min']
        max_val = state_stats[state][var]['max']
        final_pred.loc[state_mask, var] = final_pred.loc[state_mask, var].clip(
            lower=min_val, upper=max_val
        )
```

3.8.3 Scaling Post-processing

```
def apply_post_processing(df: pd.DataFrame) -> pd.DataFrame:
    """Apply Clover and Dead post-processing rules."""
    df_out = df.copy()
    if 'Dry_Clover_g' in df_out.columns:
        df_out['Dry_Clover_g'] = df_out['Dry_Clover_g'] * 0.8
    if 'Dry_Death_g' in df_out.columns:
        mask_high = df_out['Dry_Death_g'] > 20
        df_out.loc[mask_high, 'Dry_Death_g'] *= 1.1
        mask_low = df_out['Dry_Death_g'] < 10
        df_out.loc[mask_low, 'Dry_Death_g'] *= 0.9
    return df_out
```

3.9 Full Dataset Training

Since the training set is quite small, the intention was to utilize all available data and simply overfit on the training set.

```
# train_df = df[df["fold"] != fold].reset_index(drop=True)
# valid_df = df[df["fold"] == fold].reset_index(drop=True)
```

```
train_df = df
valid_df = df
```

4. Model Ensemble

4.1 Final Ensemble Scheme

The final submission uses weighted ensemble of 6 models:

Model	Weight	Features
exp-v142	0.0 (State prediction only)	BiomassModelWithStateSpecies, extracts State info
exp-v148	0.15	BiomassModelWithMetaEmbedding, uses meta features
exp-v147	0.20	BiomassModelBasic, seed=2026
exp-v174	0.25	BiomassModelBasic, random augmentation
exp-v164	0.20	BiomassModelBasic, TTA-style augmentation
exp-v177	0.20	BiomassModelBasic, adjusted loss weights

4.2 TTA Inference

Each model uses 3 TTA transforms:

1. Original view
2. Horizontal flip
3. Vertical flip

Final prediction is averaged.

4.3 Dual GPU Parallel Inference

Implemented dual GPU parallel inference for acceleration:

```
def parallel_dual_gpu_inference(dataset, model_class, model_kwargs, model_states, ...):
    """Run inference in parallel on two GPUs."""
    # Even indices -> GPU 0, Odd indices -> GPU 1
    indices_0 = list(range(0, n, 2))
    indices_1 = list(range(1, n, 2))

    with ThreadPoolExecutor(max_workers=2) as executor:
        future_0 = executor.submit(inference_single_gpu, model_0, ..., device_0)
        future_1 = executor.submit(inference_single_gpu, model_1, ..., device_1)
        results_0 = future_0.result()
        results_1 = future_1.result()

    # Merge results
    ...
```

5. Key Hyperparameters

Hyperparameter	Value
Backbone	vit_large_patch16_dinov3_qkvb
Image Size	1024x1024
Batch Size	4
Learning Rate	2e-4 (finetune to 5e-5 after warm-up)
Backbone LR Multiplier	0.1
Training Epochs	25
MoE Experts	4
MoE Hidden Dim	512
Optimizer	AdamW
Weight Decay	1e-4

6. Failed Attempts

Based on experiment records, the following methods did not improve or were unstable:

6.1 Methods That Did Not Work

Experiment	Description	Result
exp-v36	Ratio prediction	CV dropped from 0.58 to 0.43
exp-v97	Full image without splitting	CV 0.57, LB 0.52 (severe drop)
exp-v99	LR 2e-4 + 20 epochs	CV only 0.21
exp-v123	Segmentation + attention	CV 0.53, LB 0.53
exp-v168/169	Mamba Network	CV 0.80-0.89 (unstable)
exp-v180	UNet original size 1000x2000	LB 0.64 (significant drop)

6.2 Unstable Methods

Experiment	Description	Problem
Huge model (exp-v170)	vit_huge	High CV 0.95 but LB 0.72
30 epochs (exp-v161)	Increased epochs	Overfitting, Private LB dropped
Various segmentation masks	ng/xg mask	Partially effective but unstable

6.3 Lessons Learned

1. **Larger models are not always better:** Huge models easily overfit on small datasets
2. **Training epochs need control:** ~25 epochs is the optimal balance point
3. **Complex attention mechanisms have limited benefits:** Easily overfit on small datasets
4. **Mamba and other sequential model architectures are not suitable for this problem**

7. Experiment Progress

7.1 Key Milestones

Stage	Experiment	Public LB	Private LB	Key Improvement
Baseline	v1	0.57	-	448 size baseline
Model Upgrade	v24	0.65	-	vit_large_dinov3
Resolution	v32	0.69	-	512 size
Physics Loss	v42	0.69+	-	PhysicsConsistencyLoss
MoE	v44	0.69	-	Mixture-of-Experts
1024 Resolution	v108	0.73	-	1024 size + LR optimization
Full Training	v136	0.74+	-	Full dataset + Species
Seed Optimization	v147	0.75+	-	seed=2026
Augmentation	v164	0.75	-	TTA-style augmentation
Random Aug	v174	0.76	0.65	Random augmentation selection
Ensemble	Final	0.76	0.66	6-model weighted ensemble

7.2 Final Private LB Score

Based on submission records, the two final selected submissions:

- Public LB: 0.76, Private LB: 0.66 (5th Place)

8. Conclusion and Experience Sharing

8.1 Key Success Factors

1. **Extensive experiments:** Conducted 188+ experiments, fully explored the solution space
2. **Physical constraints:** Designed loss functions using physical relationships between targets
3. **Diverse ensemble:** Fusion of models with different seeds, architectures, and training strategies
4. **Data augmentation:** On small datasets, proper data augmentation is crucial

5. **Post-processing:** Post-processing strategies based on data analysis

8.2 About Submission Selection

Submission selection was very difficult in this competition, mainly because:

- Small dataset size, models easily overfit
- Distribution shift between training and test sets
- Low correlation between Public LB and Private LB

I ultimately chose the submission with the highest Public LB. Although the Private LB wasn't optimal (0.66 vs 0.65 for some failed submissions), it was relatively stable.

8.3 Luck Factor

I must admit there was some luck involved in this result:

- Saw many top players shake-up during the final standings
- Submission selection strategy happened to avoid pitfalls
- Multi-model ensemble improved stability

8.4 Acknowledgments

Special thanks to:

- The generous sharing from the Kaggle community
- Discussions and notebooks from fellow Kagglers
- CSIRO organizers for providing this meaningful competition

This solo gold medal is very important for my journey to becoming a Grandmaster! I hope this solution sharing can help everyone.

Code Repository: [2025-Kaggle-CSIRO-5th-solution](#)

Final Submission Notebook: [CSIRO-Ensemble-Models-V12](#)

Supplemental Q&A

chenwenqiang_001 · 2026-01-29T05:23:59.840000

I saw a few top solutions. the resolution is all 1024*1024 in commm.I thingk this is the key to improve

yanqiangmiffy · 2026-01-29T05:36:07.373000

Yes, the bigger resolution did work! I tried from 224,448,512,768 to 1024

chenwenqiang_001 · 2026-01-29T06:50:25.843000

dinov2 pretrained on 518518, dinov2 large with 518518 can get 0.72 in public. dinov3 pretrained on 512512, 768768. Normally dinov3 should be the best in 512 and 768. I don't know why it performs best with 1024*1024. Additionally, do you use attention pool built in timm?

Komil Parmar · 2026-01-29T07:34:05.947000

Amazing approach. Coincidentally, I also tried the same loss function, but it was too early. I did not have the rest of the things figured out till then, like the architecture itself. And it made me think that the loss function is not working for this problem, but it was just a result of unstable experiment. I should have stuck to it, or try it again when the rest was figured out.

MoE is also an amazing idea for this problem. I tried doing a Gradient Surgery from the paper: "Gradient Surgery for Multi Task Learning" to make sure all tasks are learning well, but never thought like this: If they conflict, let them conflict, just split the experts. WoW. I thought if we are using one of the N experts, wouldn't that simply perform like Dropout (except this is not random here) training only the activated expert, but now it is clear.

Just one question: Did you do some sort of analysis to see how much is each expert being activated? Actually, when I added Task and Prompt tokens, they were converging to near same representations after the training, so I had to add an orthogonality loss. I don't see any such supervision like that here. Also, did you try using more experts?

Thanks a lot for this writeup.

yanqiangmiffy · 2026-01-29T07:43:49.160000

I loaded it this way, setting the `img_size` and using average pooling.

```
self.backbone = timm.create_model(model_name, pretrained=pretrained, num_classes=0,
global_pool="avg")
```

yanqiangmiffy · 2026-01-29T07:45:54.940000

Thank you for your feedback.

```
I have another question: Have you conducted any analysis to understand the activation level of
each expert?
```

I haven't done a specific analysis on that; I used a relatively simple configuration.